Timer-series FSMs and next-state timing

Build the Review 2015 timer from its counter, shift register and controller, with a precise guide to state, next_state, nonblocking assignments and register ownership.

Kapil Tripathi | Verilog & digital design | 15 pages

CONTENTS / click a row to jump

KEEP THIS IDEA

Use state for work belonging to the current phase. Use next_state only for an intentional reaction to the chosen transition; use both to detect a one-edge entry or exit.

Tracker entries: 144, 149, 155, 160, 165, 170

Study notes by Kapil Tripathi, based on HDLBits exercises. Original questions, source references and dated verification records are retained in the notes.

ALL NOTES + ENTRY INDEX HDLBits problem collection

SERIES MAP / ENTRIES 144, 149, 155, 160, 165, 170

One timer, six connected ideas

The counter is the timing primitive. The next five exercises progressively add a shift/down register, a 1101 recognizer, a four-cycle capture controller, the complete controller, and finally the datapath.

Entry	Problem	What it contributes
144	Counter with period 1000	A precise 0...999 timebase.
149	Shift register + down counter	One register with mutually exclusive shift/count modes.
155	1101 recognizer	Find the start word and remember that it was found.
160	Enable shifting for four cycles	Turn a one-cycle event into a four-edge capture window.
165	Complete FSM	Sequence search -> capture -> count -> acknowledge.
170	Complete timer	Combine controller and datapath with exact boundaries.

Questions preserved from the work

Why can a clocked block test next_state? Is next_state one clock late? When should the block test state instead? Is the synchronous-reset structure correct?

Why can I not write assign count = 0 as well as count <= ...? What exactly are multiple drivers, and why can different tools report them differently?

Do I need to clear the four-cycle counter? Why is the fourth-bit test 3 rather than 4? How does (delay + 1) x 1000 arise without multiplication, and what does count represent?

Entry 155 appears in an earlier note too. It is repeated here deliberately because its transition into the capture phase is the timing hinge for the whole series.

ENTRY 144 / PERIOD-1000 COUNTER

Count 0 through 999, then wrap

A 10-bit register can represent 0 through 1023, so width alone does not create a modulo-1000 counter. The terminal-value comparison supplies the missing wrap rule.

```
always @(posedge clk) begin
    if (reset)
        q <= 10'd0;
    else if (q == 10'd999)
        q <= 10'd0;
    else
        q <= q + 10'd1;
end
```

Why compare with 999, not 1000?

The value visible during a cycle is the value stored after the preceding rising edge. When that stored value is 999, the next edge must write 0. Testing 1000 would expose an unwanted 1000 cycle and create a period of 1001.

Before edge	reset	Value written at edge	Visible after edge
998	0	998 + 1	999
999	0	0 (explicit wrap)	0
anything	1	0 (reset wins)	0

Synchronous reset and register ownership

Synchronous means reset is sampled only on a rising edge. Between edges, changing reset does not immediately change q. Declaring q as output reg lets the clocked process own it directly. Using an internal reg count plus assign q = count is equivalent; that continuous assignment drives q, not count.

The reset branch does not need a separate count=0 declaration elsewhere. One clocked process gives the stored value one owner, and every non-reset edge either wraps or increments it.

Source: HDLBits Exams/review2015_count1k. The open submission shows Status: Success!

ENTRY 149 / SHIFT + COUNT DATAPATH

MSB-first input still shifts toward the MSB

The first serial bit must eventually become q[3]. Appending each new bit at q[0] moves every older bit one place toward q[3].

```
always @(posedge clk) begin
    if (shift_ena)
        q <= {q[2:0], data};
    else if (count_ena)
        q <= q - 4'd1;
end
```

Capture edge	data	q after edge	Meaning
1	1	0001	First bit is newest at q[0].
2	0	0010	First bit moves to q[1].
3	0	0100	First bit moves to q[2].
4	1	1001	Serial 1001 is now q[3:0].

Why the rightmost item is data

Concatenation writes q[3:1] from the old q[2:0] and writes q[0] from the current serial input. Calling the stream MSB-first describes the order in which the four-bit number arrives; it does not mean each new bit is inserted directly into q[3].

Why q <= q - 1 wraps from 0 to 15

The destination is four bits. The mathematical result -1 is represented modulo 16, so its low four bits are 1111. An explicit 0-to-15 branch is correct and readable, but it is not required for a four-bit decrement.

There is no reset in this component's interface. Simulation may therefore show X until an enabled edge establishes a known value. The final system guarantees four shift edges before it counts, so that behavior is intentional.

Source: HDLBits Exams/review2015_shiftcount. The open submission shows Status: Success!

ENTRY 149 / PRIORITY AND OLD VALUES

Two true enables: the last assignment wins

The exercise promises that shift_ena and count_ena are not used together. Your two-if version is therefore accepted, but it still has a definite simulation meaning.

```
if (shift_ena)
    q <= {q[2:0], data};
else
    q <= q;                 // unnecessary self-assignment

if (count_ena)
    q <= q - 4'd1;          // later assignment in same process
```

All right-hand sides use the old q

Nonblocking assignments sample their right-hand sides during the active clock event and update the destination later. If both enables are 1, both calculations use the same pre-edge q, and the later scheduled write wins. The result is old q minus one; it is not the shifted value minus one.

shift_ena	count_ena	Two-if result	Explicit if/else-if result
0	0	hold	hold
1	0	shift	shift
0	1	decrement	decrement
1	1	decrement (last NBA)	shift (declared priority)

Three cleanup rules

1. Omit q <= q in a clocked process: no assignment already means hold. 2. Use if/else if when you want a visible priority. 3. Give a stored signal one procedural owner; assignment order is predictable only within that one process.

Why this is different from multiple drivers

Several nonblocking assignments to q inside one clocked block are competing assignments from one owner, resolved by source order. Assigning q from another always block or continuous assign creates another owner. That is the multiple-driver problem addressed on page 7.

ENTRY 155 / 1101 RECOGNIZER

The repeated question that unlocks the series

Five states record the longest useful suffix seen so far. The final state is sticky in this component because later exercises replace that self-loop with the four-bit capture phase.

State	Remembered suffix	Next on 0	Next on 1
SEARCH0	none	SEARCH0	SEARCH1
SEARCH1	1	SEARCH0	SEARCH11
SEARCH11	11	SEARCH110	SEARCH11
SEARCH110	110	SEARCH0	FOUND
FOUND	1101 found	FOUND	FOUND

> “Why next_state? next_state is a clock late ... is reset right?”

next_state is not another stored register here. It is combinational logic calculated from the current state and current data. On the edge that samples the final 1, next_state is FOUND before the nonblocking updates; state becomes FOUND after that edge.

```
always @(posedge clk) begin
    if (reset) begin
        state          <= SEARCH0;
        start_shifting <= 1'b0;
    end else begin
        state <= next_state;
        if (next_state == FOUND)
            start_shifting <= 1'b1;  // intentional sticky register
    end
end
```

This raises the registered output on the same edge that state enters FOUND. Testing state == FOUND in that clocked block would raise it one edge later. The cleaner Moore form is assign start_shifting = (state == FOUND): after the state update, the decode rises in the FOUND cycle without a separate output register.

The synchronous reset is correct: when reset is 1 at a rising edge, its branch wins and the next_state calculation is ignored for that state update.

Source: HDLBits Exams/review2015_fsmseq. The open submission shows Status: Success!

ENTRY 160 / FOUR-CYCLE ENABLE

Exactly four cycles, with one owner per register

The accepted counter starts at one on the reset edge while shift_ena becomes 1. Three more enabled edges advance the stored count to four; the following cycle is disabled.

Edge	Cause	count after edge	shift_ena after edge
R	reset=1	1	1
R+1	count < 4	2	1
R+2	count < 4	3	1
R+3	count < 4	4	1
R+4	count is 4	holds 4	0

> “Why can’t I add assign count = 3'd0? What does multiple drivers mean, and does it behave differently in each simulator?”

A continuous assign drives its left side all the time. The clocked block also tries to drive count after clock edges. Those are two hardware sources connected to one signal. In Verilog, a procedural destination is a variable/reg while a continuous-assignment destination is a net; mixing the two is normally illegal. If a resolved net does have two drivers, conflicting 0 and 1 values resolve to X in simulation and do not describe an ordinary FPGA register.

Pattern	Meaning	Use it?
count <= ... in one clocked block	One flip-flop bank owns count.	Yes
assign q = count	Internal count drives a separate output net q.	Yes
assign count = 0 plus count <= ...	Constant driver fights the clocked driver.	No
count <= ... in two always blocks	Two procedural owners; ordering is not a priority rule.	No

Tool messages vary—illegal reg drive, multiple drivers, unresolved signal, or synthesis failure—but the design rule does not: one signal, one owner. Initialization belongs in the reset branch of that owner.

Source: HDLBits Exams/review2015_fsmshift. The open submission shows Status: Success!

ENTRY 165 / COMPLETE CONTROLLER

Search, capture, count, notify, acknowledge

The controller combines the previous state machines but still treats the counters as an external datapath. Its outputs describe phases; done_counting returns the datapath result.

Phase	State meaning	Output/action	Exit condition
Search	Remember suffixes of 1101	All controls 0	Final 1 arrives
Capture	Receive four following bits	shift_ena=1	Four capture cycles complete
Count	Wait for external counters	counting=1	done_counting=1
Notify	Timeout is visible	done=1	ack=1

A state-per-cycle controller

```
SEARCH110: next_state = data ? BIT0 : SEARCH0;
BIT0:      next_state = BIT1;
BIT1:      next_state = BIT2;
BIT2:      next_state = BIT3;
BIT3:      next_state = COUNTING;
COUNTING:  next_state = done_counting ? WAIT_ACK : COUNTING;
WAIT_ACK:  next_state = ack ? SEARCH0 : WAIT_ACK;

assign shift_ena = (state == BIT0) || (state == BIT1) ||
                   (state == BIT2) || (state == BIT3);
assign counting  = (state == COUNTING);
assign done      = (state == WAIT_ACK);
```

This version needs more states but no separate four-cycle counter. A compressed CAPTURE state plus a two- or three-bit counter is equally valid. State count and datapath count are two representations of the same four-cycle history.

Why Moore-style output decoding is useful

Outputs are determined from the current phase, so there is one obvious cycle definition: shift_ena is high for every cycle spent in BIT0 through BIT3; counting is high while waiting; done stays high until acknowledgement. No extra output register can drift away from the state.

Source: HDLBits Exams/review2015_fsm. The open result panel shows Status: Success!

ENTRY 165 / COUNTER-COMPRESSED VERSION

Yes, the capture counter must be re-armed

Your accepted controller uses one CAPTURE state and count to remember which of the four capture cycles is active. That register must be ready at zero before every new timer transaction.

> “Do I need to make count 0 or not?”

For a controller that can return from WAIT_ACK to searching and start again, yes. If count remains 4, the next arrival in CAPTURE immediately appears complete and shift_ena never supplies four fresh cycles. Resetting count outside CAPTURE, resetting it when CAPTURE finishes, or setting it on entry are all valid—choose one clear owner and one convention.

```
always @(posedge clk) begin
    if (reset) begin
        state <= SEARCH0;
        count <= 0;
    end else begin
        state <= next_state;
        if (state == CAPTURE) begin
            if (count == 3)
                count <= 0;        // fourth cycle completed
            else
                count <= count + 1'b1;
        end else begin
            count <= 0;            // armed before next entry
        end
    end
end
assign shift_ena = (state == CAPTURE);
```

Why the terminal value is 3

With zero-based numbering, the four capture cycles are 0, 1, 2, and 3. During the cycle with count=3, shift_ena is still high and the fourth bit is sampled at its ending edge. That same edge may move state to COUNTING. Waiting for count=4 would create a fifth CAPTURE cycle.

CAPTURE cycle	count before edge	Bit sampled	state after edge
1	0	delay[3]	CAPTURE
2	1	delay[2]	CAPTURE
3	2	delay[1]	CAPTURE
4	3	delay[0]	COUNTING

CLOCKED LOGIC / THE CENTRAL CONFUSION

state is now; next_state is the chosen destination

Both names are evaluated before the nonblocking update at a rising edge. They answer different questions, so neither is universally the correct test.

Before the edge	At the edge	After NBA updates
state stores Q input has D next_state = F(Q,D)	Clocked blocks read Q, D, and F(Q,D). Every <= RHS is sampled.	state stores F(Q,D). Registered outputs receive their scheduled values.

```
always @(posedge clk) begin
    state <= next_state;

    if (state == CAPTURE)
        delay <= {delay[2:0], data};   // action of current cycle

    if (next_state == FOUND)
        found_reg <= 1'b1;             // react to chosen transition
end
```

Testing state

Use state when the action belongs to the phase the machine is already in: sample a delay bit during CAPTURE, advance a subcounter during COUNTING, or clear phase-local storage while outside that phase. This reads the pre-edge current state.

Testing next_state

Use next_state only when you intentionally want a registered reaction to the transition decision made from this edge's inputs. In the standalone recognizer, next_state==FOUND sets a sticky output on the edge that consumes the final 1. It is not a universal shortcut for avoiding a delay.

A decoded Moore output needs neither test in the sequential block: assign done = (state == WAIT_ACK). The state update itself makes the output change in the new state.

next_state is usually combinational—not a second clocked register and not inherently one cycle late.

CLOCKED LOGIC / DECISION GUIDE

When to use state, next_state, or both

Start by naming the event you mean. Then choose the expression that identifies that event exactly.

Intent	Recommended condition	Reason
Do work throughout phase X	state == X	X is the current cycle.
Decode a Moore output	assign out = (state == X)	Keeps output aligned with state.
React on transition into X	state != X && next_state == X	A true one-edge entry event.
React on transition out of X	state == X && next_state != X	A true one-edge exit event.
Register an output high whenever destination is X	next_state == X	Includes entry and any X self-loop.
Preserve a sticky event	set on entry; omit assignment otherwise	Clocked omission intentionally holds.

Why next_state == X is not an entry detector

If X self-loops, next_state remains X every cycle inside X. The condition is therefore true on entry and during the stay. Use state != X && next_state == X when exactly one entering edge matters.

Why delay capture must use state == CAPTURE

On the edge that completes 1101, current state is SEARCH110 and next_state is CAPTURE. Testing next_state would shift that final recognizer bit into delay. Four such tests would capture 1000 for an intended following delay 0001. Testing state starts on the next edge, when the first actual delay bit is present.

Reset and ownership checklist

In a synchronous-reset block, reset wins only at a rising edge. Reset state and every independently stored datapath register that must begin known. next_state can still toggle combinationally while reset is high; the reset branch ignores it. Assign next_state in the combinational block only, and state in the clocked block only.

Do not assign next_state from both blocks. Do not register next_state unless you deliberately want another pipeline stage.

ENTRY 170 / CAPTURE THE DELAY

The fourth bit uses old delay plus current data

Nonblocking assignment means delay still contains the first three captured bits while the fourth edge is being evaluated. The concatenation supplies the missing current bit immediately.

```
if (state == CAPTURE) begin
    delay <= {delay[2:0], data};
    if (bit_index == 3) begin
        bit_index <= 0;
        remaining <= {delay[2:0], data};
        subcount <= 0;
    end else begin
        bit_index <= bit_index + 1'b1;
    end
end
```

Before edge	Current data	Candidate {delay[2:0],data}	After edge
delay=0000, index=0	0	0000	delay=0000
delay=0000, index=1	0	0000	delay=0000
delay=0000, index=2	0	0000	delay=0000
delay=0000, index=3	1	0001	delay=0001; remaining=0001

> “At the third tick, if I put the completed value in countreg, do I still need next_state?”

The bit index and the state answer separate questions. bit_index==3 says this edge has the fourth delay bit, so it constructs the completed value. state==CAPTURE says the current data belongs to the delay field at all. next_state is not needed for the datapath write; combinational FSM logic independently uses bit_index==3 to leave CAPTURE on that edge.

Why the if test is == 3, not != 3

The special work—copying the completed four-bit candidate into remaining—belongs only to the fourth capture edge. Using != 3 would perform it on the first three edges and skip it precisely when the final bit arrives.

Source: HDLBits Exams/review2015_fancytimer. The open result panel shows Status: Success!

ENTRY 170 / EXACT TIMER LENGTH

Two nested counters implement (delay + 1) x 1000

remaining is not the raw clock-cycle count. It is the value displayed for a block of 1000 cycles. subcount selects the cycle within that block.

```
if (state == COUNTING) begin
    if (subcount == 10'd999) begin
        subcount <= 0;
        if (remaining != 0)
            remaining <= remaining - 1'b1;
    end else begin
        subcount <= subcount + 1'b1;
    end
end

// Leave after the final cycle of the final block.
COUNTING: next_state = (remaining == 0 && subcount == 999)
                       ? WAIT_ACK : COUNTING;
```

Loaded delay	remaining shown	Cycles per value	Total COUNTING cycles
0	0	1000	1000
1	1, then 0	1000 each	2000
5	5,4,3,2,1,0	1000 each	6000
15	15 down to 0	1000 each	16000

> “How is this block making (delay + 1) x 1000? What is count? Am I putting delay in countreg?”

Yes: remaining/countreg starts as the captured delay. Values delay, delay-1, ..., 0 form delay+1 display blocks. subcount visits 0 through 999 inside every block, giving 1000 cycles per value. Multiplication is unnecessary because the two counters express the product structurally.

The terminal comparison is made from the old pre-edge values. When remaining=0 and subcount=999, that cycle is still the thousandth cycle displaying 0; the ending edge moves to WAIT_ACK. This avoids the two common off-by-one failures: leaving at 998 or creating an extra cycle at 1000.

A 10-bit subcount is sufficient because 999 fits. A nine-bit register is not: its maximum is 511. A separate product register would need 14 bits for the maximum 16000, but this architecture never stores that product.

ENTRY 170 / REFERENCE RTL, PART 1

Controller and next-state logic

This complete version keeps the successful seven-state structure, uses one owner per stored signal, and makes every next-state path explicit.

```
module top_module (
    input clk,
    input reset,          // Synchronous reset
    input data,
    output [3:0] count,
    output counting,
    output done,
    input ack
);
    localparam SEARCH0   = 3'd0,
               SEARCH1   = 3'd1,
               SEARCH11  = 3'd2,
               SEARCH110 = 3'd3,
               CAPTURE   = 3'd4,
               COUNTING  = 3'd5,
               WAIT_ACK  = 3'd6;

    reg [2:0] state, next_state;
    reg [2:0] bit_index;
    reg [3:0] delay_shift;
    reg [3:0] remaining;
    reg [9:0] subcount;

    always @(*) begin
        case (state)
            SEARCH0:   next_state = data ? SEARCH1 : SEARCH0;
            SEARCH1:   next_state = data ? SEARCH11 : SEARCH0;
            SEARCH11:  next_state = data ? SEARCH11 : SEARCH110;
            SEARCH110: next_state = data ? CAPTURE : SEARCH0;
            CAPTURE:   next_state = (bit_index == 3) ? COUNTING : CAPTURE;
            COUNTING:  next_state = (remaining == 0 && subcount == 999)
                                    ? WAIT_ACK : COUNTING;
            WAIT_ACK:  next_state = ack ? SEARCH0 : WAIT_ACK;
            default:   next_state = SEARCH0;
        endcase
    end
```

Why the default branch matters

Every legal state assigns next_state, and default recovers from an unused or unknown encoding. This is complete combinational logic: it does not intentionally remember an earlier next_state value and therefore does not infer a latch.

The SEARCH11 self-loop preserves overlap. After 111, the most useful suffix is still 11, so a following 01 can complete the pattern.

ENTRY 170 / REFERENCE RTL, PART 2

Datapath, outputs and a final audit

The sequential block performs work for the current phase; continuous Moore decodes expose the phase after each state update.

```
    always @(posedge clk) begin
        if (reset) begin
            state       <= SEARCH0;
            bit_index   <= 0;
            delay_shift <= 0;
            remaining   <= 0;
            subcount    <= 0;
        end else begin
            state <= next_state;

            if (state == CAPTURE) begin
                delay_shift <= {delay_shift[2:0], data};
                if (bit_index == 3) begin
                    remaining <= {delay_shift[2:0], data};
                    bit_index <= 0;
                    subcount  <= 0;
                end else begin
                    bit_index <= bit_index + 1'b1;
                end
            end else begin
                bit_index <= 0;
            end

            if (state == COUNTING) begin
                if (subcount == 10'd999) begin
                    subcount <= 0;
                    if (remaining != 0)
                        remaining <= remaining - 1'b1;
                end else begin
                    subcount <= subcount + 1'b1;
                end
            end
        end
    end

    assign count    = remaining;
    assign counting = (state == COUNTING);
    assign done     = (state == WAIT_ACK);
endmodule
```

Final audit

Check	What to confirm
Sampling	The final 1 of 1101 is not the first delay bit.
Capture	Exactly four edges shift MSB-first; index 3 completes the value.
Timing	subcount spans 0...999 for every remaining value, including 0.
Outputs	counting and done decode current state; done holds until ack.
Reset	State and all independently stored datapath values reset on the edge.
Ownership	Each register is written by one clocked block; next_state by one combinational block.

Verification used for this note

The builder checks the 998,999,0,1 wrap; MSB-first capture of 1001; four-bit underflow; entry to CAPTURE after 1101; the wrong 1000 value produced by consuming the recognizer's final 1; and every delay value 0 through 15 for exact duration and 1000-cycle count plateaus.

The six open HDLBits pages showed successful submissions. That platform evidence establishes acceptance; the local model above independently checks the timing claims printed in this note.